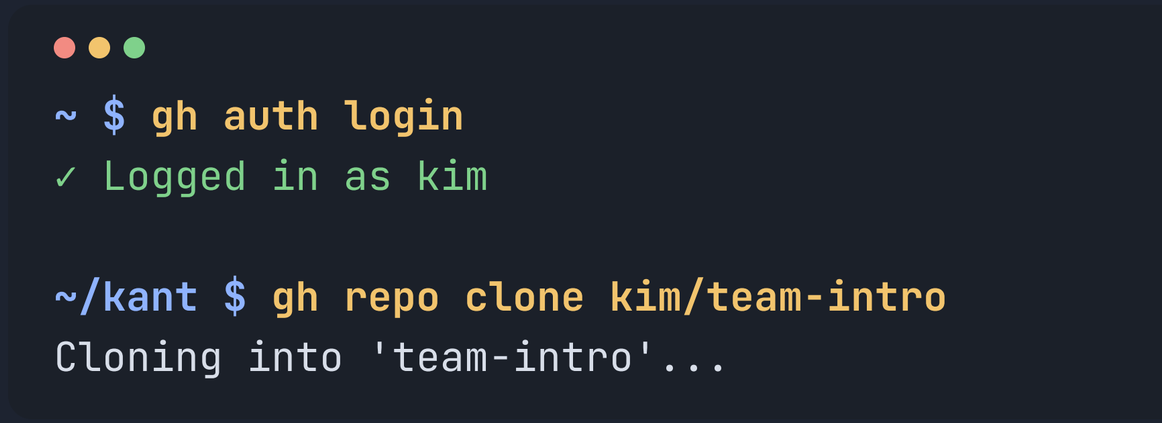
KANT AX/AI 엔지니어 트랙 · 11

터미널에서 GitHub 다루기

gh로 로그인하고
레포 만들고 지우기



오늘의 순서

1 **설치하고 로그인하기** — 6–15쪽
`gh auth login, 권한 목록, git도 같이 로그인`

2 **이미 있는 레포 연결하기** — 16–19쪽
`gh repo clone, view, browse, list`

3 **새 레포 만들기** — 20–24쪽
`gh repo create 뒤 git push origin main`

4 **레포 지우기와 권한 추가** — 25–30쪽
`delete가 막힐 때 gh auth refresh`

5 **자주 쓰는 gh 명령** — 31–35쪽
`이슈, PR 만들기와 합치기`

3–5쪽은 들어가기, 36–37쪽은 막힐 때와 한 장 요약이에요.

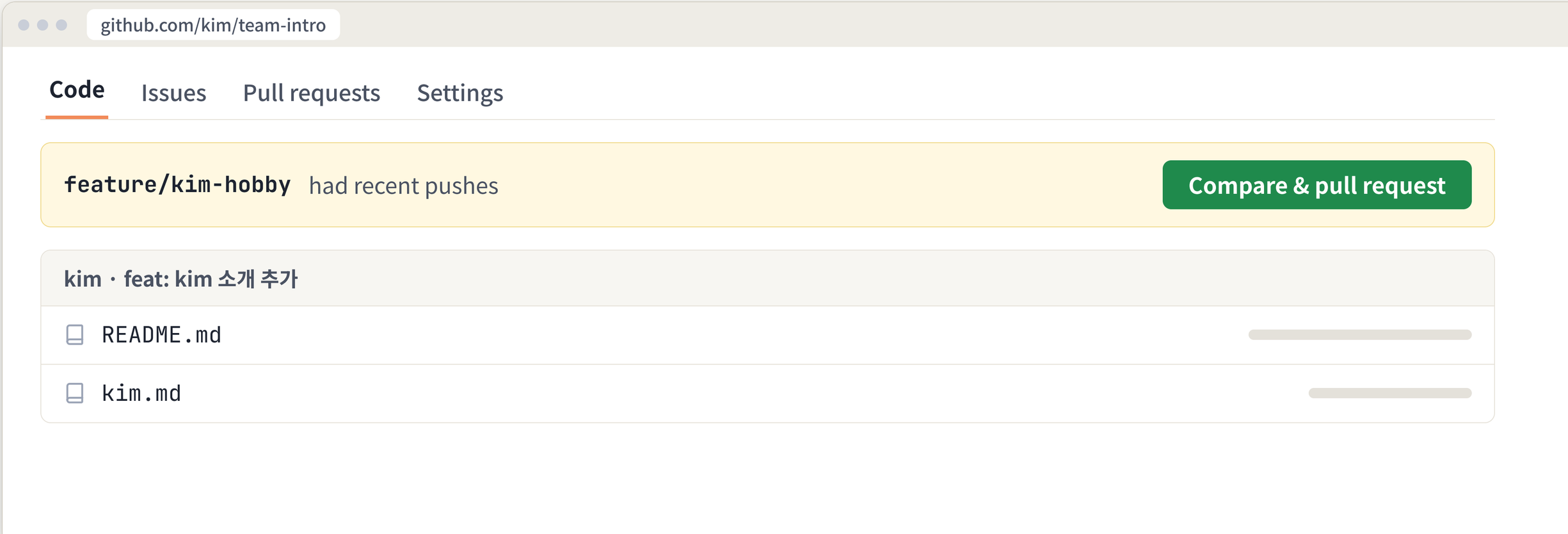
들어가기

push는 터미널에서, PR 버튼은 브라우저에서 눌렀어요

```
~/kant/team-intro $ git push origin feature/kim-hobby
…
 * [new branch]      feature/kim-hobby -> feature/kim-hobby
```

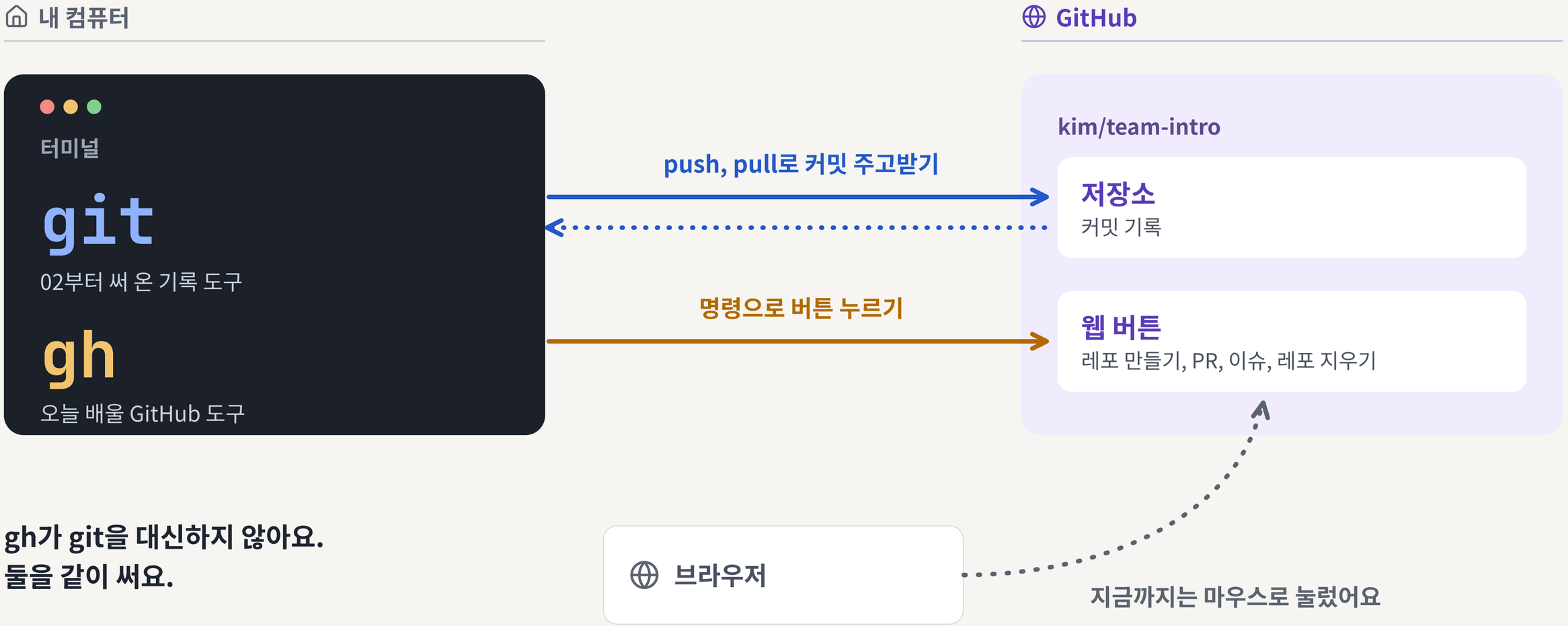
```
~/kant/team-intro $ gh pr create
```

오늘은 이 버튼도 터미널에서 눌러요.



들어가기

git은 커밋을 주고받고, gh는 웹 버튼을 맡아요



gh가 git을 대신하지 않아요.
둘을 같이 써요.

들어가기

웹에서 누르던 버튼마다 gh 명령이 하나씩 있어요

GitHub 웹에서	gh 대상 할 일
로그인	`gh auth login`
Code › 주소 복사 뒤 clone	`gh repo clone kim/team-intro`
New repository	`gh repo create my-notes …`
Settings › Delete this repository	`gh repo delete kim/my-notes`
New issue	`gh issue create`
Compare & pull request	`gh pr create`

gh 다음에 대상, 그다음에 할 일을 적어요.

1. 설치하고 로그인하기

brew나 winget으로 설치하고 gh --version으로 확인해요

```
# Mac
~ $ brew install gh

# Windows (Git Bash)
~ $ winget install --id GitHub.cli -e

~ $ gh --version
gh version 2.89.0 (2026-03-26)
https://github.com/cli/cli/releases/tag/v2.89.0
```

09에서 uv를 설치한 Homebrew

끝나면 Git Bash를 다시 열어요

숫자는 설치한 날마다 달라요

1. 설치하고 로그인하기

gh auth status로 로그인했는지 물어봐요

```
~ $ gh auth status
You are not logged into any GitHub hosts. To log in, run: gh auth login
```

출력 끝에 다음에 칠 명령까지 적혀 있어요.



1. 설치하고 로그인하기

질문 1 · 어떤 GitHub을 쓰나요?

```
~ $ gh auth login
? Where do you use GitHub?  [Use arrows to move, type to filter]
> GitHub.com
  Other
```

Enter

github.com 웹사이트를 쓰니 고른 그대로 Enter예요. Other는 회사 전용 GitHub일 때 골라요.

질문 1	질문 2	질문 3	질문 4
GitHub.com			

질문 2 · git은 어떤 주소로 주고받을까요?

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host?  [Use arrows to move, type to filter]
> HTTPS
  SSH
```

Enter

02부터 써 온 https:// 주소 방식이에요. SSH는 열쇠 파일을 따로 만들어야 해서 이 과정에서는 쓰지 않아요.

질문 1	질문 2	질문 3	질문 4
GitHub.com	HTTPS		

질문 3 · git도 같이 로그인할까요?

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? (Y/n)
```

Enter

Yes로 하면 git push 할 때도 gh 로그인을 써요. 대문자 Y가 기본값이라 Enter만 눌러도 Yes예요.

질문 1	질문 2	질문 3	질문 4
GitHub.com	HTTPS	Yes	

질문 4 · 어떻게 로그인할까요?

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI?  [Use arrows to move, type to filter]
> Login with a web browser
  Paste an authentication token
```

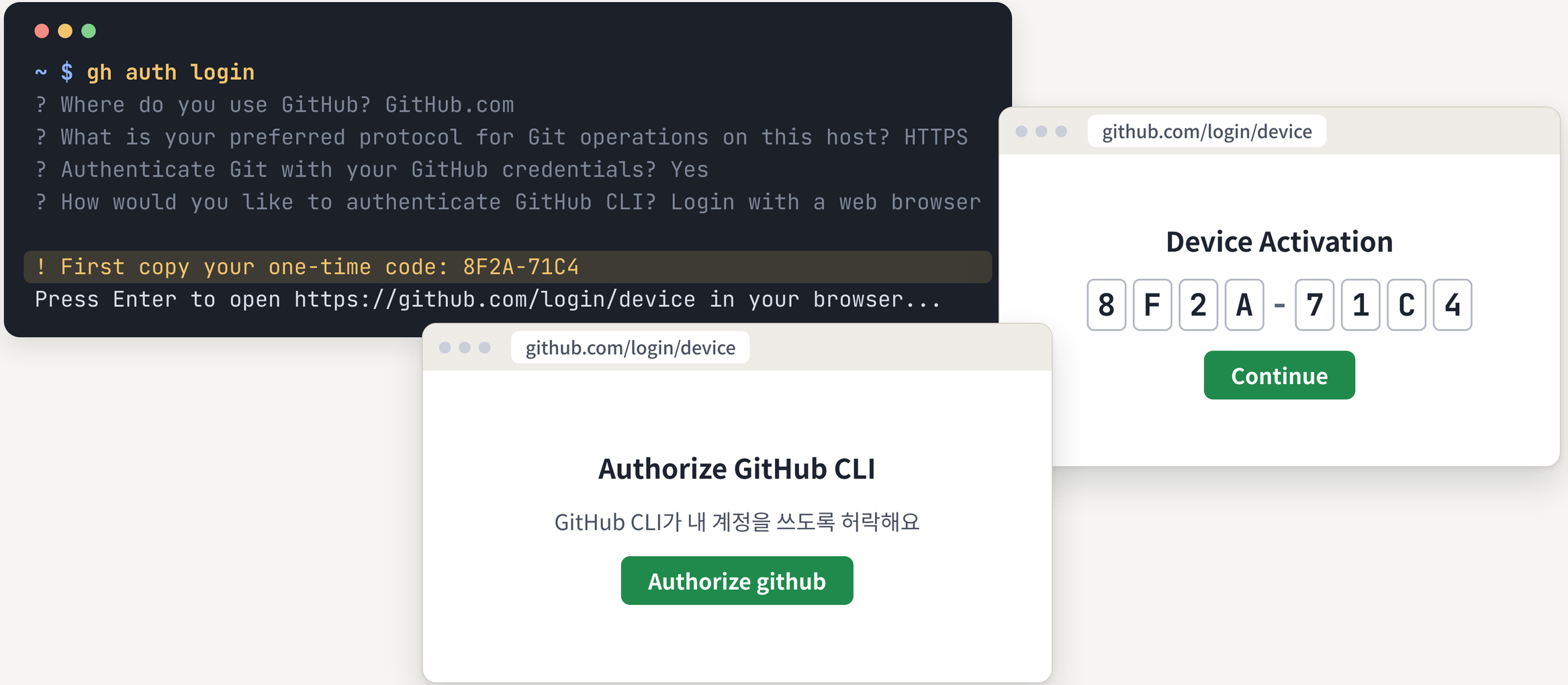
Enter

브라우저로 로그인해요. 토큰을 직접 만들어 붙여 넣는 방법은 쓰지 않아요.

질문 1	질문 2	질문 3	질문 4
GitHub.com	HTTPS	Yes	web browser

1. 설치하고 로그인하기

터미널에 나온 코드를 브라우저에 넣어요



1. 설치하고 로그인하기

Logged in as 다음에 내 아이디가 보이면 성공이에요

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 8F2A-71C4
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ Logged in as kim
```

브라우저 허락 끝

HTTPS로 저장

Mac, Windows에서는 보통 안 나와요

내 GitHub 아이디

1. 설치하고 로그인하기

로그인하면 출입증(토큰)과 열 수 있는 문(권한) 목록이 생겨요

```
~ $ gh auth status
github.com
  ✓ Logged in to github.com account kim (/home/kim/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_*********
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

출입증(토큰), 앞 네 글자만 보여요

출입증으로 열 수 있는 문(권한)

출입증 (토큰)

repo

read:org

gist

workflow

이 목록에 없는 일은 못 해요. 4파트에서 여기에 한 칸이 늘어요.

1. 설치하고 로그인하기

질문 3의 Yes 덕분에 git push도 gh 로그인을 써요

```
~ $ cat ~/.gitconfig

…
[pull]
    rebase = false
[credential "https://github.com"]
    helper =
    helper = !/usr/local/bin/gh auth git-credential
…
```

gh가 넣은 줄, 고치지 않아요

`git push origin main` → git이 gh에게 출입증을 받아요 → GitHub에 바로 올라가요

02의 첫 push에서 뜨던 로그인 창이 이제 안 떠요.

2. 이미 있는 레포 연결하기

gh repo clone은 주소 대신 주인/이름만 적어요

git으로 받을 때 (06)

```
git clone https://github.com/kim/team-intro.git
```

gh로 받을 때

```
gh repo clone kim/team-intro
```

```
~/kant $ gh repo clone kim/team-intro
Cloning into 'team-intro'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (6/6), 510 bytes | 510.00 KiB/s, done.
```

git clone과 같은 출력

2. 이미 있는 레포 연결하기

gh로 받은 폴더도 git으로 받은 폴더와 똑같아요

```
~/kant $ cd team-intro
~/kant/team-intro $ git remote -v
origin    https://github.com/kim/team-intro.git (fetch)
origin    https://github.com/kim/team-intro.git (push)

~/kant/team-intro $ git log --oneline
1baff03 (HEAD -> main, origin/main, origin/HEAD) feat: kim 소개 추가
a2a187e docs: README 추가
```

origin도 그대로

06의 커밋 그대로

push, pull도 지금처럼 `git push origin main`, `git pull origin main`으로 해요.

반대로 git clone으로 받은 폴더에서도 gh를 바로 쓸 수 있어요.

2. 이미 있는 레포 연결하기

레포 폴더 안에서는 gh repo view, gh browse만 쳐도 돼요

```
~/kant/team-intro $ gh repo view
kim/team-intro
No description provided

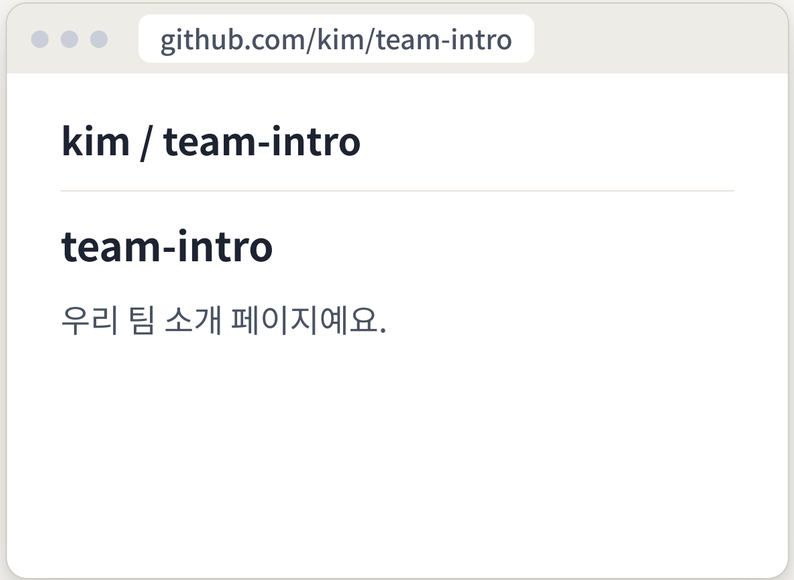

  # team-intro

  우리 팀 소개 페이지예요.


View this repository on GitHub: https://github.com/kim/team-intro
```

origin을 보고 알아냈어요

```
~/kant/team-intro $ gh browse
Opening https://github.com/kim/team-intro
in your browser.
```



2. 이미 있는 레포 연결하기

gh repo list는 어느 폴더에서 쳐도 내 레포 목록을 보여 줘요

```
~ $ gh repo list

Showing 2 of 2 repositories in @kim

NAME              DESCRIPTION  INFO    UPDATED
kim/hello-git                 public  about 3 minutes ago
kim/team-intro                public  about 3 minutes ago
```

clone, delete에 적을 주인/이름은 이 칸에서 복사해요.

3. 새 레포 만들기

웹에서 네 단계로 하던 일을 gh는 한 줄로 해요

웹에서 (02)

1. GitHub에서 New 누르기
2. 이름 적고 Create repository
3. 주소 복사
4. `git remote add origin 주소`

```
~/kant/my-notes $ gh repo create my-notes
    --public --source=. --remote=origin
```

만들고 origin에 연결하는 데까지 gh가 해요.
push는 지금처럼 git으로 해요.

3. 새 레포 만들기

먼저 내 컴퓨터에 저장소와 커밋 하나를 만들어요

```
~/kant $ mkdir my-notes
~/kant $ cd my-notes
~/kant/my-notes $ git init
Initialized empty Git repository in /home/kim/kant/my-notes/.git/
~/kant/my-notes $ echo '# my-notes' > README.md
~/kant/my-notes $ git add README.md
~/kant/my-notes $ git commit -m 'docs: README 추가'
[main (root-commit) f5c3a4c] docs: README 추가
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

이 커밋을 GitHub에 올려요

3. 새 레포 만들기

gh repo create로 레포를 만들고 origin까지 연결해요

```
gh repo create my-notes --public --source=. --remote=origin
```

- `my-notes` — GitHub에 만들 레포 이름
- `--public` — 누구나 보기 (나만: `--private`)
- `--source=.` — 지금 폴더(.)를 레포로 써요
- `--remote=origin` — origin 이름으로 연결해요

```
~/kant/my-notes $ gh repo create my-notes --public --source=. --remote=origin
✓ Created repository kim/my-notes on github.com
  https://github.com/kim/my-notes
✓ Added remote https://github.com/kim/my-notes.git
```

GitHub에 만들었어요

origin으로 연결했어요

`--push` 옵션은 붙이지 않아요. push는 다음 장에서 직접 쳐요.

3. 새 레포 만들기

push는 git으로 하고, gh repo list로 확인해요

```
~/kant/my-notes $ git remote -v
origin    https://github.com/kim/my-notes.git (fetch)
origin    https://github.com/kim/my-notes.git (push)
~/kant/my-notes $ git push origin main
…
To https://github.com/kim/my-notes.git
 * [new branch]      main -> main
```

로그인 창 없이 올라갔어요

```
~ $ gh repo list
NAME
kim/hello-git
kim/my-notes
kim/team-intro
```

질문 3에서 Yes를 고른 덕분이에요.

3. 새 레포 만들기

옵션이 생각나지 않으면 gh repo create만 쳐요

```
~/kant $ gh repo create
? What would you like to do?  [Use arrows to move, type to filter]
> Create a new repository on github.com from scratch
  Create a new repository on github.com from a template repository
  Push an existing local repository to github.com
```

앞 장에서 한 일

화살표로 고르고 Enter를 누르면 폴더, 이름, 공개 여부를 차례로 물어요.

마지막에 push 할지 물으면 No를 고르고 `git push origin main`으로 직접 올려요.

4. 레포 지우기와 권한 추가

지운 레포는 되돌릴 수 없어요

github.com/kim/my-notes

GitHub의 kim/my-notes

~~커밋 기록과 README~~	레포 안의 파일과 기록
~~이슈와 PR~~	토론과 리뷰 기록
~~스타와 포크 연결~~	다른 사람이 누른 것

내 컴퓨터

`~/kant/my-notes`

폴더는 남아요

모두 사라지고, GitHub에는 휴지통이 없어요.

연습용으로 만든 my-notes만 지워요. team-intro처럼 팀이 쓰는 레포는 지우지 않아요.

4. 레포 지우기와 권한 추가

처음 gh repo delete는 권한이 없어서 막혀요

```
~ $ gh repo delete kim/my-notes

? Type kim/my-notes to confirm deletion: kim/my-notes
HTTP 403 Must have admin rights to Repository. (https://api.github.com/repos/kim/my-notes)
This API operation needs the "delete_repo" scope. To request it, run:
  gh auth refresh -h github.com -s delete_repo
```

실수하지 않게 이름을 한 번 더 쳐요

허락할 수 없다는 뜻

칠 명령까지 알려 줘요

출입증 (토큰)

repo | read:org | gist | workflow | delete_repo ?

고장이 아니에요. 처음 로그인할 때는 지우기 권한을 일부러 빼고 받아요.

4. 레포 지우기와 권한 추가

gh auth refresh로 출입증에 지우기 권한을 더해요

출입증 (토큰)

`repo` `read:org` `gist` `workflow` `delete_repo`

1파트 gh auth status의 Token scopes 줄이에요.

```
gh auth refresh -h github.com -s delete_repo
```

- `gh auth refresh` — 출입증을 새로 받아요
- `-h github.com` — 어느 GitHub인지 (-h = host, 도움말 아님)
- `-s delete_repo` — 더할 권한(scope)

질문 네 개는 다시 묻지 않고 출입증만 다시 받아요.

4. 레포 지우기와 권한 추가

브라우저에서 한 번 더 허락하면 권한이 더해져요

```
~ $ gh auth refresh -h github.com -s delete_repo

! First copy your one-time code: 8F2A-71C4
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
```

권한 추가 끝



새로 더해지는 권한이에요.
코드 넣기는 로그인 때와 같아요.

4. 레포 지우기와 권한 추가

권한 목록에 delete_repo가 생겼어요

출입증 (토큰)

repo | read:org | gist | workflow | delete_repo

이제 지우기를 다시 해 봐요.

```
~ $ gh auth status
github.com
  ✓ Logged in to github.com account kim (/home/kim/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_*********
  - Token scopes: 'delete_repo', 'gist', 'read:org', 'repo', 'workflow'
```

4. 레포 지우기와 권한 추가

다시 gh repo delete를 치면 지워져요

```
~ $ gh repo delete kim/my-notes

? Type kim/my-notes to confirm deletion: kim/my-notes
✓ Deleted repository kim/my-notes
```

지워졌어요

```
~ $ gh repo list
Showing 2 of 2 repositories in @kim
```

my-notes가 없어요

```
NAME             DESCRIPTION  INFO    UPDATED
kim/hello-git                public  about 3 minutes ago
kim/team-intro               public  about 3 minutes ago
```

```
~/kant/my-notes $ git push origin main
fatal: repository
'https://github.com/kim/my-notes.git/'
not found
```

내 폴더는 남아 있어요. GitHub 쪽이 없어서 push하면 not found가 나요.

폴더도 필요 없으면 01의 `rm -r`로 지워요.

5. 자주 쓰는 gh 명령

gh issue create로 할 일을 이슈로 남겨요

```
gh issue create --title "소개 페이지에 사진 넣기" --body "각자 사진 한 장씩"
                        |                                  |
                      제목                               내용
```

```
~/kant/team-intro $ gh issue create --title "소개 페이지에 사진 넣기" --body "각자 사진 한 장씩 넣어요"
Creating issue in kim/team-intro

https://github.com/kim/team-intro/issues/1   ← 만든 이슈 주소

~/kant/team-intro $ gh issue list
Showing 1 of 1 open issue in kim/team-intro

ID  TITLE                  LABELS  UPDATED
#1  소개 페이지에 사진 넣기          less than a minute ago   ← 이슈 번호
```

--title, --body를 빼면 gh가 하나씩 물어봐요.

5. 자주 쓰는 gh 명령

브랜치를 push한 뒤 gh pr create로 PR을 열어요

```
~/kant/team-intro $ git checkout -b feature/kim-hobby
Switched to a new branch 'feature/kim-hobby'
~/kant/team-intro $ echo '취미: 등산' >> kim.md
~/kant/team-intro $ git add kim.md
~/kant/team-intro $ git commit -m 'feat: kim 취미 추가'
[feature/kim-hobby 8039736] feat: kim 취미 추가
 1 file changed, 1 insertion(+)
~/kant/team-intro $ git push origin feature/kim-hobby
…
 * [new branch]      feature/kim-hobby -> feature/kim-hobby

~/kant/team-intro $ gh pr create --title "feat: kim 취미 추가" --body "kim.md에 취미를 적었어요"
Creating pull request for feature/kim-hobby into main in kim/team-intro

https://github.com/kim/team-intro/pull/2
```

먼저 push

PR 주소

5. 자주 쓰는 gh 명령

gh pr list, gh pr view로 PR을 터미널에서 봐요

```
~/kant/team-intro $ gh pr list
Showing 1 of 1 open pull request in kim/team-intro

ID  TITLE                BRANCH              CREATED AT
#2  feat: kim 취미 추가  feature/kim-hobby  less than a minute ago

~/kant/team-intro $ gh pr view 2
feat: kim 취미 추가 kim/team-intro#2
Open • kim (kim) wants to merge 1 commit into main from feature/kim-hobby • less than a minute ago
+1 -0 • No checks


  kim.md에 취미를 적었어요


View this pull request on GitHub: https://github.com/kim/team-intro/pull/2
```

이슈 #1 다음이라 2번

칠한 줄은 kim이 커밋 1개를 main에 합치고 싶다는 뜻이에요.

5. 자주 쓰는 gh 명령

gh pr merge로 합치고, main은 git pull로 받아 와요

```
~/kant/team-intro $ gh pr merge 2 --merge
✓ Merged pull request kim/team-intro#2 (feat: kim 취미 추가)

~/kant/team-intro $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
~/kant/team-intro $ git pull origin main
…
   1baff03..5487d25  main       -> origin/main
Updating 1baff03..5487d25
Fast-forward
```

합쳤어요

```
~/kant/team-intro $ git log --oneline
5487d25 Merge pull request #2 …
8039736 feat: kim 취미 추가
1baff03 feat: kim 소개 추가
a2a187e docs: README 추가
```

--merge는 08에서 본 Merge 버튼 세 가지 중 Create a merge commit이에요.

5. 자주 쓰는 gh 명령

다른 명령은 gh --help에서 찾아요

```
~ $ gh --help
Work seamlessly with GitHub from the command line.

USAGE
  gh <command> <subcommand> [flags]

CORE COMMANDS
  auth:          Authenticate gh and git with GitHub
  browse:        Open repositories, issues, pull requests, and more in the browser
  codespace:     Connect to and manage codespaces
  gist:          Manage gists
  issue:         Manage issues
  org:           Manage organizations
  pr:            Manage pull requests
  project:       Work with GitHub Projects.
  release:       Manage releases
  repo:          Manage repositories
  …
```

칠한 줄이 오늘 쓴 묶음이에요.

한 단계 더 보려면

```
~ $ gh repo --help
~ $ gh pr create --help
```

묶음 뒤에 붙이면 할 일 목록, 할 일 뒤에 붙이면 옵션 목록이 나와요.

정리

gh에서 막히면 이렇게 풀어요

이런 글이 보이면	이렇게 해요
`gh: command not found` 설치 직후 터미널이 gh를 몰라요	**터미널 껐다 켜기**
`You are not logged into any GitHub hosts` 아직 로그인 전이에요	`gh auth login`
`needs the "delete_repo" scope` 지우기 권한이 없어요	`gh auth refresh -h github.com -s delete_repo`
`failed to run git: fatal: not a git repository` 레포 폴더 밖에서 쳤어요	**pwd 뒤 cd 레포폴더**
Where should we push the 'feature/...' branch? 브랜치를 push하기 전에 PR을 만들었어요	**Ctrl+C 뒤 git push origin 브랜치**
브라우저가 안 열려요 주소를 직접 열어요	`github.com/login/device`

정리

한 장 요약

로그인

`gh auth login`
로그인

`gh auth status`
로그인 상태, 권한

`gh auth refresh -s 권한`
권한 더하기

레포

`gh repo clone 주인/이름`
받기

`gh repo view`
지금 레포 보기

`gh repo list`
내 레포 목록

`gh repo create 이름 …`
만들기

`gh repo delete 주인/이름`
지우기

이슈와 PR

`gh issue create`
이슈 만들기

`gh issue list`
이슈 목록

`gh pr create`
PR 만들기

`gh pr list / view`
PR 보기

`gh pr merge 번호`
PR 합치기

커밋과 push는 git으로, 웹 버튼은 gh로 해요.

레포 지우기는 되돌릴 수 없어요. 지우기 전에 주인/이름을 한 번 더 확인해요.